

Session 06 - Large Language Models (LLM)

It seems a lot of this lecture is based a lot on these articles:

- [The GPT-3 Architecture, on a Napkin](#)
- [Transformers...Attention is all you need!](#)

To understand LLMs Architecture better this interactive visualisation is a great, even if it is a bit out of the scope of the lecture: [LLM Visualization, by Brendan Bycroft](#)

Large Language Models (LLMs)

Large Language Models (LLMs) are a class of machine learning models that can generate natural language by predicting the next word in a sequence based on previous context. This predictive ability enables a variety of applications, from chatbots to code completion and beyond.

Development of LLMs

A historical overview reveals three main branches in the evolution of language models:

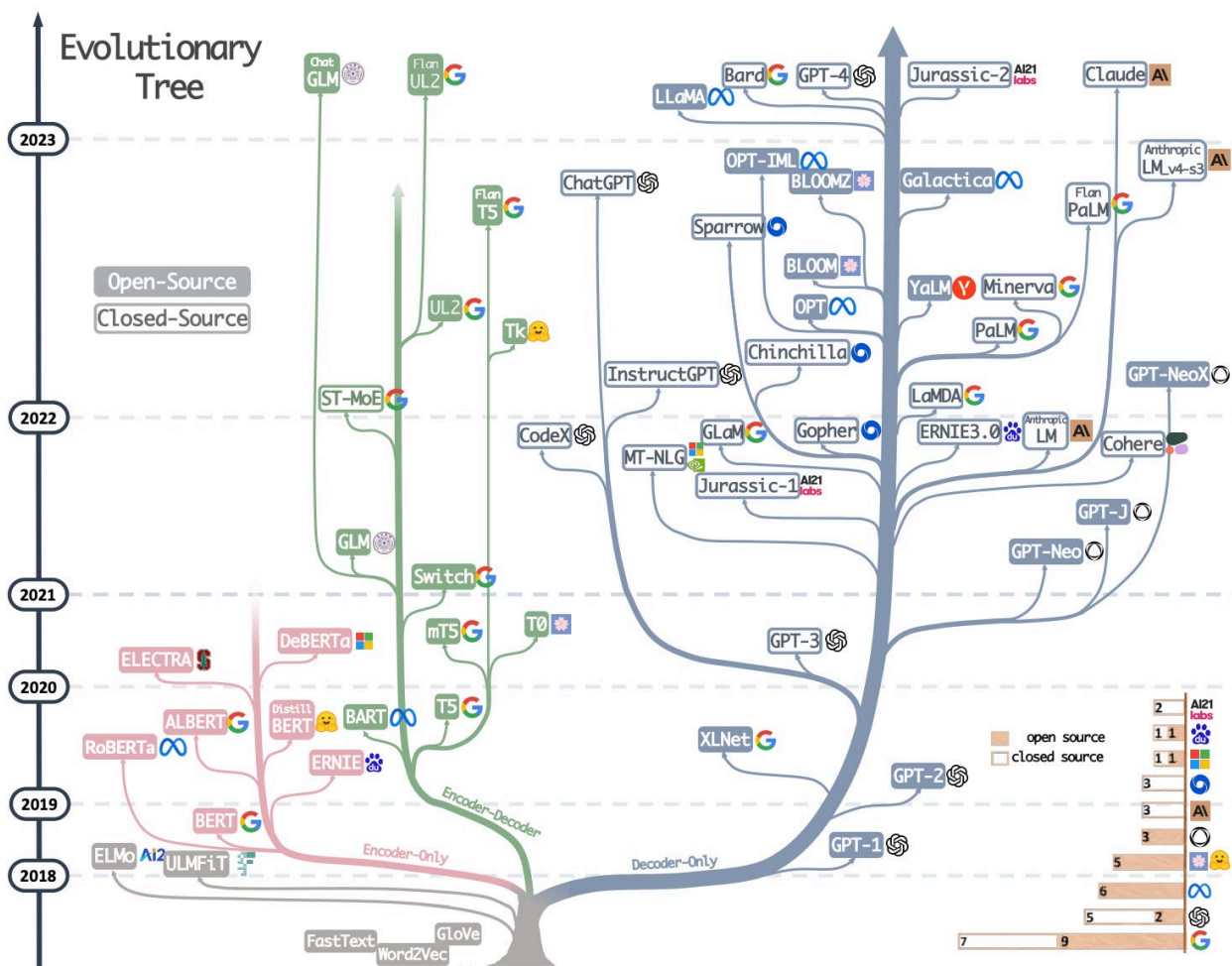


Fig. 1. The evolutionary tree of modern LLMs traces the development of language models in recent years and highlights some of the most well-known models. Models on the same branch have closer relationships. Transformer-based models are shown in non-grey colors: decoder-only models in the blue branch, encoder-only models in the pink branch, and encoder-decoder models in the green branch. The vertical position of the models on the timeline represents their release dates. Open-source models are represented by solid squares, while closed-source models are represented by hollow ones. The stacked bar plot in the bottom right corner shows the number of models from various companies and institutions.

1. **Encoder-only models** like **BERT**: Best suited for understanding tasks such as classification, entity recognition, and sentiment analysis.
2. **Decoder-only models** like **GPT**: Optimized for generative tasks such as writing, summarization, and code generation.
3. **Encoder-decoder models** like **BART** and **T5**: Hybrid models designed for translation, summarization, and sequence-to-sequence tasks.

Earlier steps in this evolution include models like **Word2Vec**, **GloVe**, **ELMo**, and **ULMFiT**, which laid the groundwork by focusing on word embeddings and contextual language representations.

🔗 **Think of the model types as tools:**

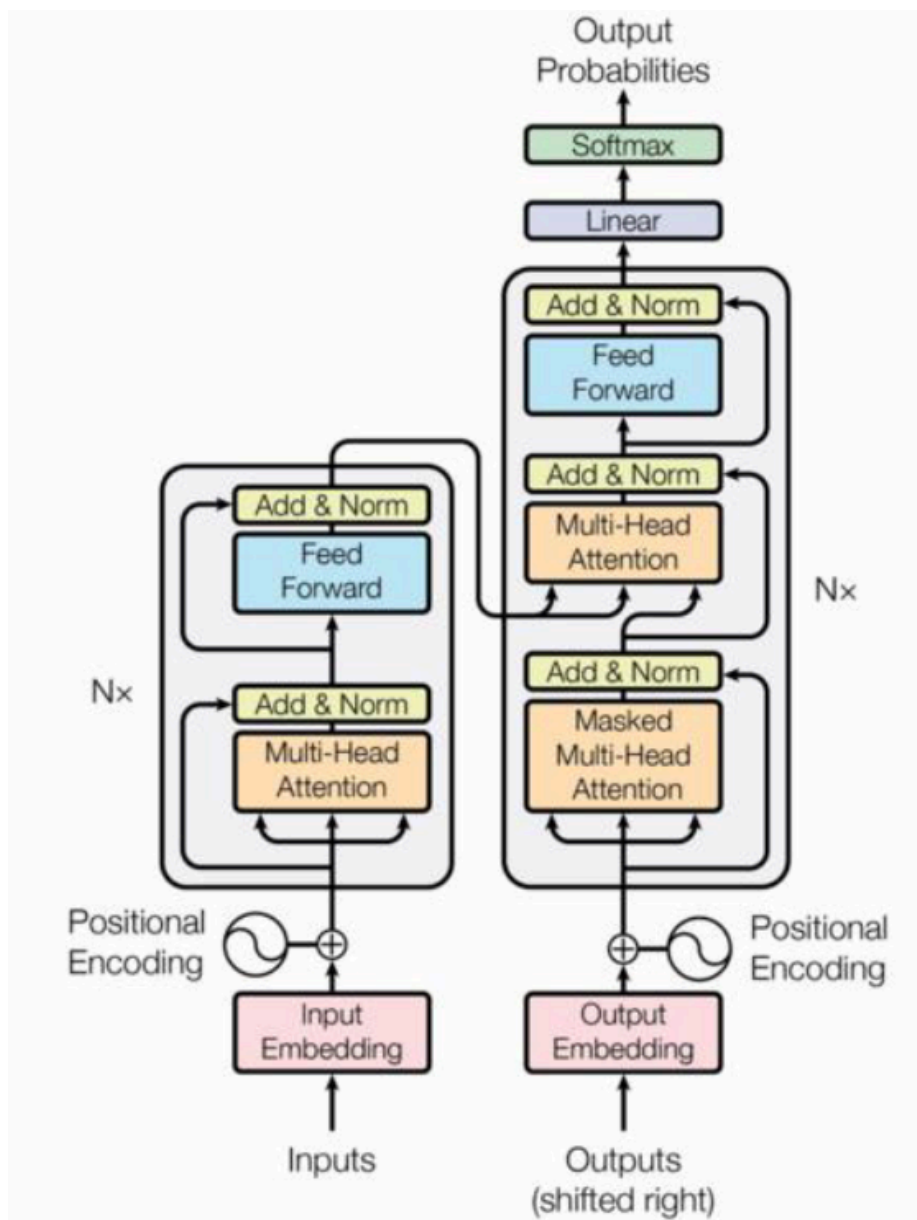
- Encoder-only: "understand this"
- Decoder-only: "write this"
- Encoder-decoder: "understand and then write"

Transformer Architecture

📖 **Vaswani et al., 2017 – *Attention is All You Need***

This foundational paper introduced the transformer architecture, enabling massive improvements in parallelization and sequence modeling.

Transformers revolutionized NLP by using attention mechanisms to process input data in parallel rather than sequentially (as done in RNNs).



Encoder (left side)

The encoder's job is to **read and understand the input sequence**, and transform it into a rich contextual representation.

Input → Embedding + Positional Encoding

- **Input tokens** (e.g., words) are first turned into **word embeddings**, i.e., numerical vectors.
- **Positional Encoding** is added to retain word order, which is not captured by the model otherwise.

🔗 **Positional encoding adds a sense of time or order, kind of like coordinates for each word in a sentence.**

N Layers (e.g., 6 times repeated)

Each layer has two sub-layers:

1. **Multi-Head Self-Attention**
 - Each word attends to all other words to gather contextual meaning.
2. **Add & Norm**
 - A **residual connection** (Add) is followed by **Layer Normalization** (Norm).

- Ensures stability during training and helps with gradient flow.

🔗 **Multi-head means multiple attention heads look at the sequence from different perspectives in parallel.**

3. Feedforward Network (FFN)

- Applies two linear transformations with a ReLU activation in between.

4. Again, **Add & Norm**

Passing Data to Decoder

The encoder outputs a **contextual embedding** for each input token (a matrix of vectors). These go into the decoder's **Encoder-Decoder Attention**, allowing the decoder to look back at the original input while generating output.

Decoder (right side)

The decoder **generates the output sequence one word at a time**, using both past outputs and encoder information.

Nx Layers

Each decoder layer consists of **three** sub-layers:

1. Masked Multi-Head Self-Attention

- Only attends to earlier positions to preserve autoregressive prediction.

🔗 **This ensures that the decoder cannot "cheat" by looking at future words.**

2. Encoder-Decoder Attention

- Attends to the encoder's output.
- Integrates input context to help generate the correct next word.

🔗 **This is where translation or understanding of input context happens.**

3. Feedforward Network + Add & Norm (same as in encoder)

Output: Linear & Softmax

After the Nx decoder blocks, the output goes through:

- **Linear Layer:** Projects the decoder output to the size of the vocabulary.
- **Softmax Layer:** Converts scores into probabilities for each word in the vocabulary.

🔗 **The Softmax tells use "What is the most likely next word?"**

It outputs a probability distribution over the vocabulary.

DataFlow

Input Tokens → Embedding + Positional Encoding → Encoder (Nx Layers) → Encoder Output → Decoder
(Masked Self-Attention + Encoder Attention + FFN) → Linear → Softmax → Output Probabilities

🔗 Why does the decoder need two types of attention?

One is **masked self-attention** for autoregressive output. The other is **encoder-decoder attention** to incorporate the input sequence's context.

Key components:

- **Self-Attention Mechanism:** Helps the model focus on relevant words regardless of their position.
- **Feedforward Neural Networks (FFNNs):** Applied independently to each position.
- **Positional Encoding:** Adds order awareness since transformers lack recurrence.

🔗 The Transformer architecture

It scales better and captures long-range dependencies more effectively than RNNs or LSTMs.

For more detail: <https://chiranthancv95.medium.com/transformers-attention-is-all-you-need-8de139e0fe9e>

Attention Mechanism

Imagine the sentence: *"The model needs a lot of patience and even more glue."*

🔍 Attention Mechanism:

Attention calculates how much each word in a sentence should influence another when creating the next output word. In this sentence, the word "glue" might influence the interpretation of "model" toward a physical model (like a plane), depending on context.

Example Attention Table:

	model	patience	glue
model	(self-reference, weight ~0.3)	0.2	0.5
patience	0	(self ~1)	0
glue	0	0	(self ~1)

🔗 Attention is like asking:

"What other parts of this sentence should I look at to better understand this word?"

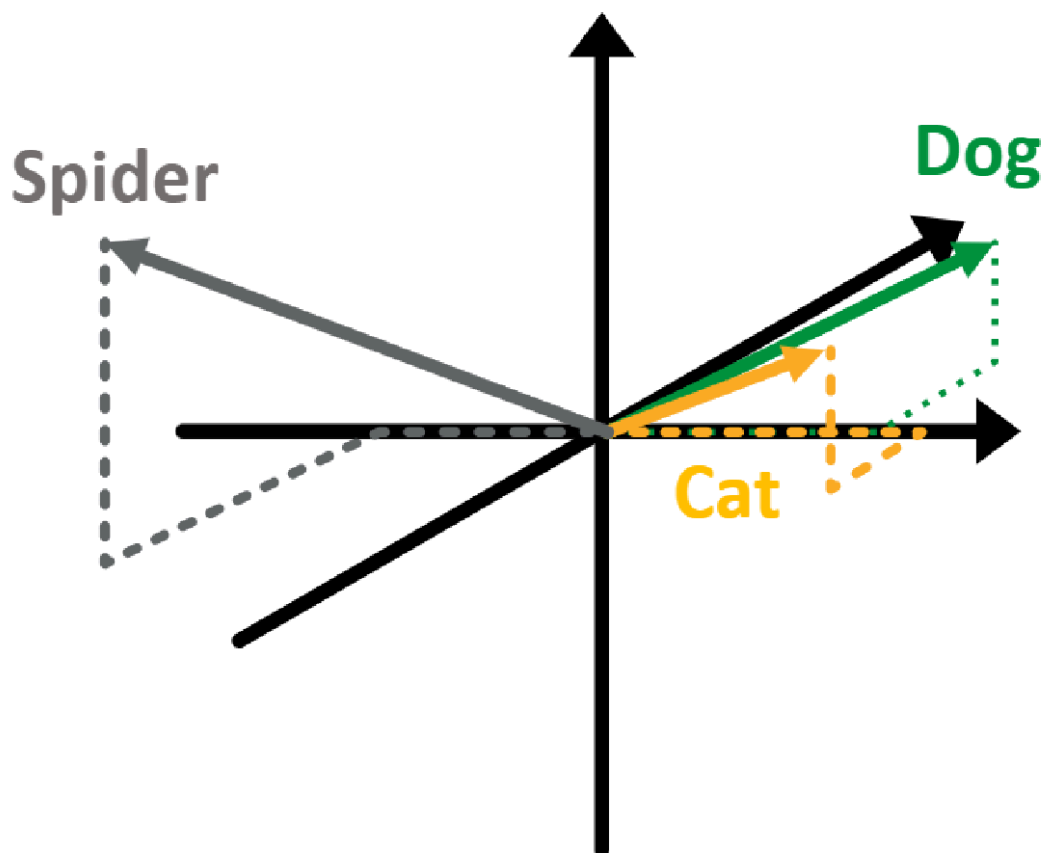
🔗 Why does training transformers require so much time and compute?

Each attention head calculates a matrix of interactions between all tokens, which grows quadratically with input length. A context window of 1000 tokens implies an attention matrix of 1 million values per layer.

Embeddings and Semantics

Before a model can reason about words, it converts them into vectors called **embeddings**.

These **Word Embeddings** place words as vectors in a high-dimensional space such that their **semantic relationships** are reflected in their **geometric distances** and **orientations**.



The vector-space might have a meaning for each axis like this:

- **X-axis = Domesticated**
- **Y-axis = Animal**
- **X-axis = Affectionate**

Embeddings:

Embeddings represent words in high-dimensional space such that semantic relationships are preserved. For instance, the vector difference between "king" and "queen" is similar to that between "man" and "woman".

 Words like "cat" and "dog" will lie close in embedding space, while unrelated words like "banana" and "justice" will be far apart.

Word embeddings can capture **semantic relationships** as **vector directions**. A famous example is how we can take the vector between man and woman and add it to the vector of king and reach the vector of queen:

```
`vec(king) - vec(man) + vec(woman) ≈ vec(queen)`
```

This works because:

- The vector from **man** → **woman** encodes the **gender dimension**.
- Applying this **same vector offset** to **king** shifts it along the same gender axis, landing near **queen**.

🔗 It shows that embeddings don't just store meaning, they capture **relationships between meanings as directions in space**.

❓ How do embeddings capture similarity?

Embeddings use the **distributional hypothesis**: "You shall know a word by the company it keeps." If two words appear in similar contexts, their embeddings are adjusted during training to become similar.

How LLMs Generate Text

LLMs work through **next-word prediction**, evaluating which word is most likely to follow a sequence:

Prompt	Model predictions (with probabilities):
"It may seem like ..."	- "magic" → 32% - "rain" → 12% - "a" → 24% - "nothing" → 15%

These probabilities are passed through a **Softmax function** to normalize them.



🔍 Softmax & Temperature:

- **Softmax** converts raw scores into probabilities.
- **Temperature** controls randomness. Lower temperature makes output more deterministic; higher allows for more diversity.

🔗 Temperature is like a creativity dial

We can crank it up for poems, turn it down for factual answers.

Temperature Explained: [# Why Does My LLM Have A Temperature?](#)

? How does temperature affect LLM output?

Higher temperature increases randomness and creativity; lower temperature prioritizes the most probable word.

Attention Mechanism

Great youtube video to understand the attention mechanism [Attention in transformers, step-by-step - by Grant Sanderson aka 3blue1brown](#)

The **attention mechanism** is a fundamental part of transformer-based language models. It allows the model to dynamically determine **which other words in a sequence are relevant** to a given word when making predictions or building word representations.

i Attention Mechanism

Attention is used in both **text classification** and **text generation** (language modeling). In classification, attention helps the model understand the meaning of a sentence to assign a label. In generation, attention helps determine what the most likely next word should be, based on the context.

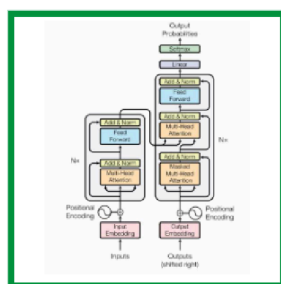
Wee look at the sentence:

“This model needs patience and a lot of glue.”

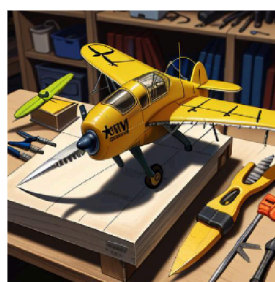
This sentence is ambiguous! Particularly the word “**model**.” It could refer to:

- A **generic model** (like a mathematical or software model)
- A **physical model**, like a plane or sculpture

So how does the LLM figure out the right interpretation?



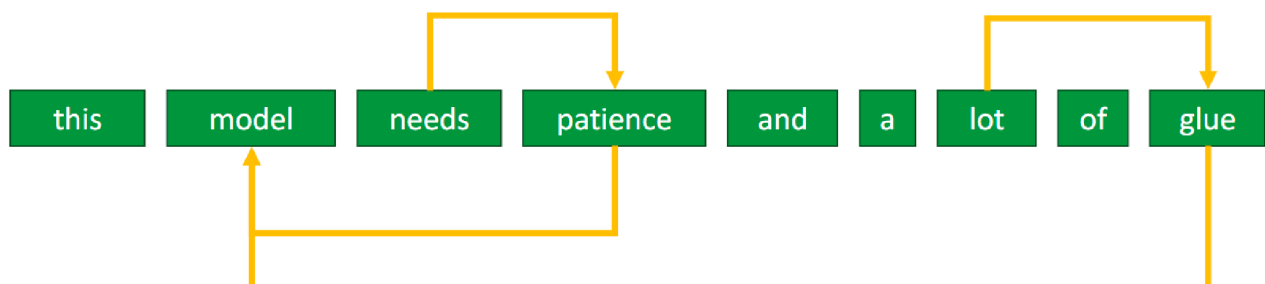
model



model



model



1. Attention Assigns Relevance Weights

- Each word in the sentence is turned into a vector. For each word, the model calculates **how much attention to pay to every other word**.
- The attention mechanism computes scores between “model” and all the other words in the sentence and gives them a score based on how **aligned their vectors are** in the embedding space. The more aligned, the **higher the attention weight**.
 - These scores (after normalization via softmax) determine how much influence each word has on the **final representation** of “model”.

2. Context Helps Resolve Meaning

- In the sentence, here's how attention likely behaves:

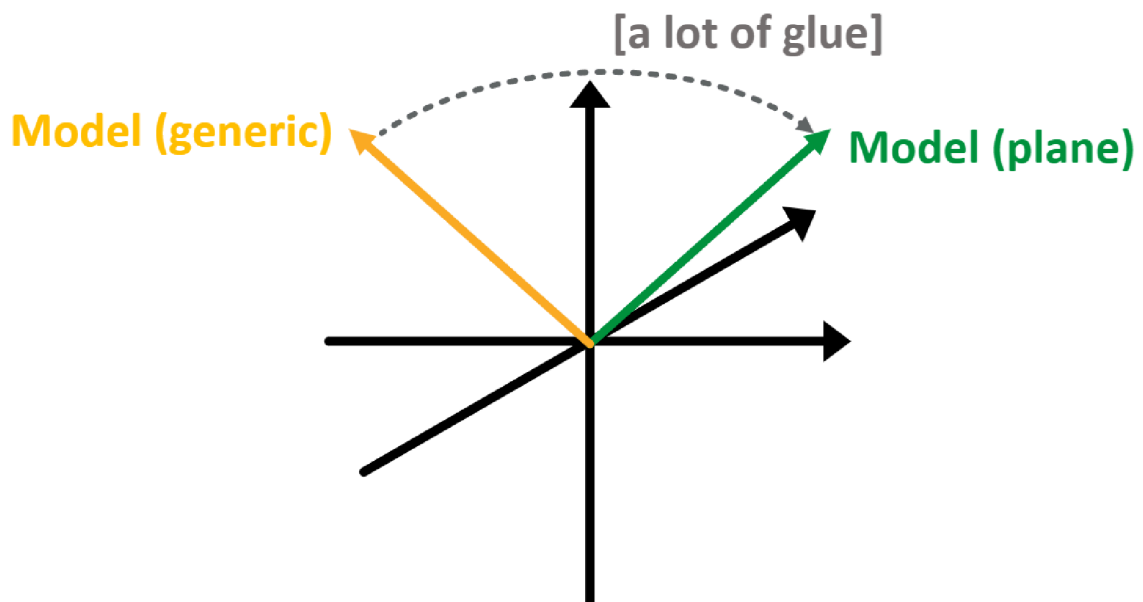
From Word	Pays Attention To	Reason
needs	patience	Verbs like “needs” often highlight the object/requirement
patience	model	“Patience” is required by the “model” here the dependency is backward
a lot of glue	glue	Emphasizes the material; this cluster likely has internal attention
glue	model	“Glue” refers back to the object that requires it; again, “model”

This is a **bidirectional flow**

While processing “glue”, the model recalls “model” to figure out what the glue is needed for.

3. Going from “Generic Model” to “Plane Model”

- LLMs use **word embeddings** where words have multiple possible meanings (polysemy). The base word “model” might sit near many clusters in vector space: one for **scientific models**, one for **fashion models**, one for **physical models** like **airplanes**.



4. So how does the LLM shift “model” from a **generic** meaning to the **physical (plane)** one?

- Attention and **contextualization**:
 - The model processes the sentence multiple times (through stacked layers).
 - Each layer **updates the embedding** of each word based on attention to others.
 - Initially, “model” might be placed near the generic cluster.

- After integrating attention from “**glue**” and “**patience**”, its meaning shifts toward the physical interpretation (e.g., “model airplane”).

“A lot of glue” is highly **specific** and **concrete**. Glue is **rarely associated with abstract models** (mathematical, conceptual) but is **highly associated with physical models**, like plastic kits or wood crafts, so its alignment is closest with the vector of a model (plane) - in the graphic pointing to the top right corner in the vector space

🔗 Attention doesn't just “look around”

It **reweights word meaning based on neighboring words**. In this case, it acts like a semantic lens, refocusing “model” based on the dominant signal from “glue”.

Attention vs. Classification vs. Prediction

- In **text classification**, attention helps find which parts of the input are important for assigning a label (e.g., “positive” or “news”).
- In **text prediction**, attention builds **context-aware representations** of each token, which are used to guess the next most likely word.

❓ How does attention help resolve ambiguity in words?

Attention connects a word like “model” with context words like “glue” to disambiguate its meaning. It allows the model to adapt the embedding dynamically, depending on sentence context.

- **Masked multi-head self-attention** in transformers allows the model to attend only to previous tokens during training, preserving the autoregressive property needed for language generation.
- The **feed-forward network** then processes each token's contextualized representation independently, enabling non-linear transformations that enhance the model's capacity to capture complex patterns.

***Autoregressive** means the model generates each token based on the tokens that came before it. It predicts the next word step-by-step, never “looking ahead” to future words. This way, it can generate coherent sequences like sentences or paragraphs, one token at a time.*

***Non-linear transformations** are mathematical operations inside the model (like applying activation functions such as ReLU or GELU) that let it learn complex patterns beyond simple straight-line relationships. Without them, the model would only be able to represent simple linear functions, limiting its ability to understand and generate natural language.*

- Transformer Visual Guide: https://dugas.ch/artificial_curiosity/GPT_architecture.html

Context and Memory in LLMs

LLMs rely heavily on the **context window**, the number of tokens they can “see” at once. All contextual knowledge is encoded into the final token of a prompt during inference.

Context Capabilities:

Model	Max Tokens	Parameter Count
GPT-3	2k	~17 Billion
GPT-3.5	4.5k	>175 Billion (est.)
GPT-4	8–32k	~1.75 Trillion (est.)
GPT-4o	128k	???

🔥 **More context tokens = better memory of past input, but also exponentially more computation.**

Multimodal Interaction

Modern LLMs like GPT-4o can interact across modalities processing text, images, audio, and video.

Video Input → Frame Analysis (CLIP) → Audio Extraction → Speech Recognition → Prompt Fusion (Middleware) → LLM (GPT-4) → Response

Each modality (video, audio, text) is first converted into text, because today's LLMs are primarily text-native. That's why **translation layers** are essential for multimodal systems.

Running LLMs Locally vs. via API

Many LLMs can be accessed through APIs (e.g., OpenAI, Anthropic), but increasingly, local models are gaining traction.

Benefits of Local Models:

Factor	Vendor API	Local Model
Data Privacy	Limited	High
Cost	Ongoing	One-time
Introspection	Minimal	Full control
Setup Complexity	Low	Higher

🔥 **Tools like** `ollama` **enable easy use of local models (e.g.,** `ollama run llama3` **)**

Real-World Use Cases and Interfaces

LLMs are already widely used in:

- **Autocomplete** (e.g., GitHub Copilot)
- **Conversational agents** (e.g., ChatGPT)
- **Browser tools**

